

DynMIRTO — Schedule Explorer

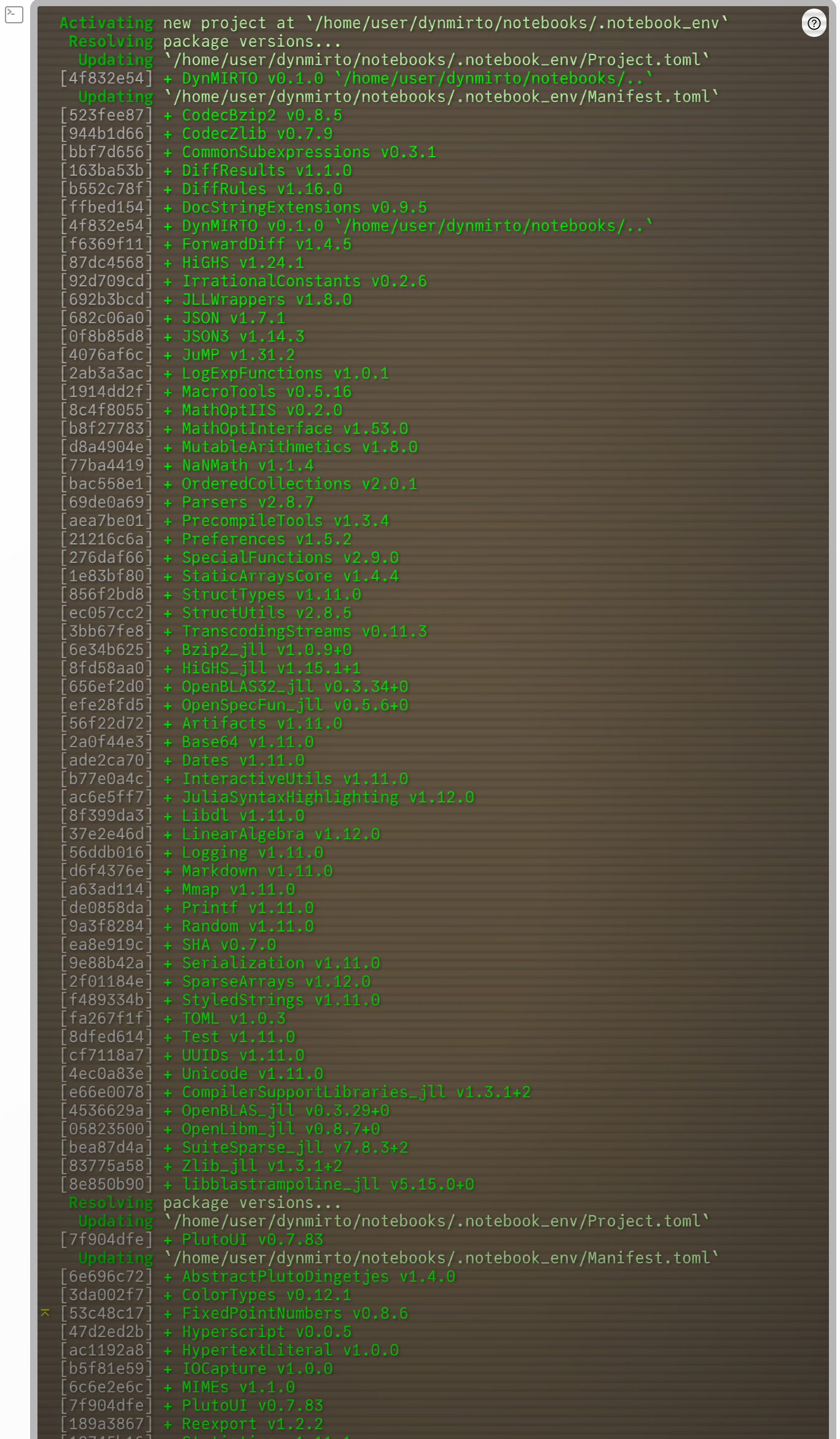
Interactive view of the v2 multi-period scheduler: tanks whose inventory evolves over a horizon via scheduled receipts and blending draws, planned against per-period demand.

Move the sliders below to change demand, the size of each scheduled receipt, the octane spec, or how much butane the tank can hold — the whole horizon re-solves and the tank-level grid and per-period recipes update live.

Uses the `DynMIRTO` package straight from `src/` (via `Pkg.develop`), so it always reflects the current code.

```
1 md"""
2 # DynMIRTO – Schedule Explorer
3
4 Interactive view of the v2 multi-period scheduler: tanks whose inventory
5 evolves over a horizon via scheduled receipts and blending draws, planned
6 against per-period demand.
7
8 Move the sliders below to change demand, the size of each scheduled
9 receipt, the octane spec, or how much butane the tank can hold – the whole
10 horizon re-solves and the tank-level grid and per-period recipes update
11 live.
12
13 Uses the `DynMIRTO` package straight from `src/` (via `Pkg.develop`), so
14 it always reflects the current code.
15 """
16
```

```
1 begin
2   import Pkg
3   Pkg.activate(joinpath(@__DIR__, ".notebook_env"))
4   Pkg.develop(path=joinpath(@__DIR__, ".."))
5   Pkg.add(["PlutoUI"])
6   using DynMIRTO
7   using PlutoUI
8 end
9
```



► ScheduledProduct("REG", "Regular Unleaded", Dict{4 ⇒ 500.0, 2 ⇒ 500.0, 3 ⇒ 500.0, 1 ⇒ 500.0})

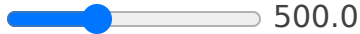
```
1 begin
2   base_components, base_tanks, base_products, base_receipts, base_periods =
3     load_schedule(joinpath(@__DIR__, "..", "data", "examples",
4                           "schedule_example.json"))
5   base_product = only(base_products)
6 end
```

Controls

```
1 md"""## Controls"""
2
```

Demand per period (same target every period)

```
1 md"**Demand per period** (same target every period)"
2
```



```
1 @bind demand_level Slider(300.0:25.0:900.0; default=500.0, show_value=true)
2
```

Regular unleaded — minimum RON

```
1 md"**Regular unleaded – minimum RON**"
2
```



```
1 @bind ron_min Slider(85.0:0.5:95.0; default=87.0, show_value=true)
2
```

Reformat receipt at T-REF, period 3 (volume units)

```
1 md"**Reformat receipt** at T-REF, period 3 (volume units)"
2
```



```
1 @bind receipt_ref Slider(0.0:100.0:3000.0; default=1500.0, show_value=true)
2
```

FCC receipt at T-FCC, period 2 (volume units)

```
1 md"**FCC receipt** at T-FCC, period 2 (volume units)"
2
```



```
1 @bind receipt_fcc Slider(0.0:100.0:2000.0; default=800.0, show_value=true)
2
```

Alkylate receipt at T-ALK, period 3 (volume units)

```
1 md"**Alkylate receipt** at T-ALK, period 3 (volume units)"
2
```



```
1 @bind receipt_alk Slider(0.0:100.0:2000.0; default=600.0, show_value=true)
2
```

T-BUT tank capacity (max volume it can hold)

```
1 md"**T-BUT tank capacity** (max volume it can hold)"
2
```



```
1 @bind but_capacity_max Slider(100.0:50.0:1000.0; default=400.0, show_value=true)
2
```

► ScheduleResult(:optimal, Dict(("REG", 2) => PeriodRecipe(2, "REG", Dict(… more), Dict(… more),

```
1 begin
2     function override_tank(tank)
3         tank.id == "T-BUT" && return Tank(tank.id, tank.component_id, tank.capacity_min,
4             but_capacity_max, min(tank.initial_inventory, but_capacity_max))
5         return tank
6     end
7     tanks = override_tank.(base_tanks)
8
9     receipts = Dict{Tuple{String,Int},Float64}{
10         ("T-REF", 3) => receipt_ref,
11         ("T-FCC", 2) => receipt_fcc,
12         ("T-ALK", 3) => receipt_alk,
13     }
14
15     demand = Dict{t => demand_level for t in base_periods}
16     product = ScheduledProduct(base_product.id, base_product.name, demand;
17         price=base_product.price,
18         specs=merge(base_product.specs, Dict(
19             :RON => PropertySpec(; min=ron_min, blend_rule=:index),
20         )),
21         eligible_components=base_product.eligible_components)
22
23     result = optimize_schedule(base_components, tanks, [product], receipts,
24         base_periods)
25 end
```

Tank levels at end of each period

Pink = at or near its floor (capacity_min).

tank	period 1	period 2	period 3	period 4
T-BUT max 400.0, floor 0.0	110.6	71.2	39.4	0.0
T-REF max 3000.0, floor 200.0	874.4	548.8	1723.2	1397.6
T-FCC max 2500.0, floor 100.0	1672.1	2344.2	2216.4	2088.5
T-ALK max 2000.0, floor 100.0	892.9	885.7	1471.0	1463.9

```
1 let
2   if result.status == :optimal
3     periods_sorted = sort(collect(base_periods))
4     header_cells = join(["<th>period $t</th>" for t in periods_sorted])
5     rows = join([
6       begin
7         cells = join([
8           begin
9             level = result.tank_levels[(tank.id, t)]
10            frac = tank.capacity_max > 0 ? level / tank.capacity_max : 0.0
11            near_floor = isapprox(level, tank.capacity_min; atol=1e-2) ||
12              level <= tank.capacity_min + 1e-6
13            cell_bg = near_floor ? "#f8d7d5" : "#f7f7f7"
14            bar_bg = near_floor ? "#c0392b" : "#4c78a8"
15            ""<td style="background:$cell_bg;">
16              <div style="font-family:monospace;font-size:0.85em;">$(round(level; digits=1))</div>
17              <div style="background:#e3e3e3;border-radius:3px;overflow:hidden;width:80px;min-width:80px;">
18                <div style="width:$(max(100*frac, 3));background:$bar_bg;height:10px;"></div>
19              </div>
20            </td>""
21          end
22          for t in periods_sorted
23        ])
24        "<tr><td><b>$(tank.id)</b><br><span style='font-size:0.8em;color:#666;'>max $(round(tank.capacity_max;digits=0)),
25          floor $(round(tank.capacity_min;digits=0))</span></td>$cells</tr>"
26      end
27      for tank in tanks
28    ])
29    HTML("""
30    <h4>Tank levels at end of each period</h4>
31    <p style="font-size:0.85em;color:#666;">Pink = at or near its floor
32    (capacity_min).</p>
33    <table>
34    <tr><th>tank</th>$header_cells</tr>
35    $rows
36    </table>
37    """)
38  else
39    diag_items = join(["<li>$d</li>" for d in result.diagnostics])
40    HTML("<h4 style='color:#b00;'>Infeasible</h4><ul>$diag_items</ul>")
41  end
42 end
```

Per-period recipe (product: Regular Unleaded)

period	ALK	BUT	FCC	REF	RON	cost
period 1	7.1	39.4	127.9	325.6	95.71	27735.04
period 2	7.1	39.4	127.9	325.6	95.71	27735.04
period 3	14.7	31.9	127.9	325.6	95.77	27848.11
period 4	7.1	39.4	127.9	325.6	95.71	27735.04

```
1 let
2   if result.status == :optimal
3     periods_sorted = sort(collect(base_periods))
4     comp_ids = sort([c.id for c in base_components])
5     header_cells = join(["<th>$(cid)</th>" for cid in comp_ids]) * "<th>RON</th>
6     <th>cost</th>"
7     rows = join([
8       begin
9         recipe = result.recipes[(product.id, t)]
10        vol_cells = join(["<td>$(round(get(recipe.volumes, cid, 0.0); digits=1))
11          </td>" for cid in comp_ids])
12        ron = round(get(recipe.resulting_properties, :RON, NaN); digits=2)
13        "<tr><td><b>period $t</b></td>$vol_cells<td>$ron</td>
14          <td>$(round(recipe.cost; digits=2))</td></tr>"
15      end
16      for t in periods_sorted
17    ])
18    HTML("""
19    <h4>Per-period recipe (product: $(product.name))</h4>
20    <table>
21    <tr><th>period</th>$header_cells</tr>
22    $rows
23    </table>
24    """)
25  else
26    HTML("<h4>Per-period recipe</h4><p>Scenario is infeasible with the current
27    sliders – see diagnostics above.</p>")
28  end
29 end
```



```
=====
Period 1
=====

Product: Regular Unleaded (REG)
Demand: 500.00

Recipe:
  ALK          7.15  ( 1.43%)
  BUT         39.38  ( 7.88%)
  FCC        127.88  (25.58%)
  REF        325.59  (65.12%)

Total cost: 27735.04

Resulting quality:
  RON      95.7117 [min=87.0000 max=-] OK
  RVP       9.0000 [min=- max=9.0000] OK (binding)
  benzene_pct 1.3000 [min=- max=1.3000] OK (binding)
  sulfur_ppm  80.0000 [min=- max=80.0000] OK (binding)

Tank levels at end of period 1:
  T-BUT      110.62 [min=0.00 max=400.00]
  T-REF      874.41 [min=200.00 max=3000.00]
  T-FCC     1672.12 [min=100.00 max=2500.00]
  T-ALK       892.85 [min=100.00 max=2000.00]

=====
Period 2
=====

Product: Regular Unleaded (REG)
Demand: 500.00

1  begin
2    io = IOBuffer()
3    print_schedule_report(io, [product], tanks, result, base_periods)
4    Text(String(take!(io)))
5  end
6
```